

MMIP_week2

Q1

初步架構建立 (claude)

```
python中main.py /Data/UCI_Credit_Card.csv, /Codes/Quiz1.py
我選擇一份二元分類資料集:import kagglehub
# Download latest version
path = kagglehub.dataset_download("fedesoriano/stroke-prediction-dataset")
• 使用 Scikit-Learn 基於任一 Machine Learning Classification Algorithm 建立分類模型。
• 將資料切分為 Training Dataset 與 Validation Dataset。
• 請進行適當的 Feature Scaling →讓數值在各欄位不要有太大的變化
• 取得模型輸出的分類分數或機率，並自行決定 Classification Threshold。
• 基於選定 Threshold 建立 Confusion Matrix，並計算：Accuracy、Precision、Recall、F1-Score
• 微調 Threshold 重新建立 Confusion Matrix，再次計算：Accuracy、Precision、Recall、F1-Score
然後再使用另一種 Machine Learning Algorithm 重新建立模型，並找出合適的 Threshold。
• 使用相同 Validation Dataset 比較兩個模型的：Accuracy、Precision、Recall、F1-Score
• 說明兩個模型在 Precision 與 Recall 上的差異，以及哪一類錯誤較多。
```

Created 2 files, shared files >

我寫好了 `Codes/Quiz1.py` 和 `main.py`，但這個環境沒有網路，所以程式我沒有跑過，下面沒有實際數值。

專案結構

增進程式易讀性 (gpt)

```
請讓程式易讀性提高，我是用ipynb
import kagglehub
# https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset
path = kagglehub.dataset_download("fedesoriano/stroke-prediction-dataset")
df = pd.read_csv(glob.glob(os.path.join(path, "*.csv"))[0])

print("資料維度:", df.shape)
print(df["stroke"].value_counts())
print(df.head())
Show more ▾
```

可以。你現在的程式功能其實已經完整，主要問題是全部塞在少數幾個 cell 裡，對 Colab / `.ipynb` 報告來說會比較難讀。

我建議改成「一個步驟一個 Cell」，而且把變數名稱稍微統一。這樣你之後寫報告時，也可以直接用每個 Markdown 標題當章節。

Cell 1 — Import Libraries

Q2

初步架構建立 (claude)

python中main.py /Data/UCI_Credit_Card.csv, /Codes/Quiz2.py
我需要進行適當的 Feature Scaling →讓數值在各欄位不要有太大的變化 • 自行選擇 TensorFlow 或 PyTorch，建立一個可用的 Multi-Layer Perceptron • 有程式碼實際展示預測 Validation Dataset 的一筆 Sample • 實際訓練至少：50 Epochs，並繪製：Training Loss、Validation Loss • 說明超參數，例如：Epoch、Batch Size、Learning Rate、Optimizer、Hidden Layer / Neuron 數量

Ran a command, created a file, shared files >

这份 `Quiz2.py` 用 PyTorch 完成了你要求的所有部分：

- **Feature Scaling:** `StandardScaler` 只在训练集上 `fit`，再套用到验证集，避免 data leakage，也让 `LIMIT_BAL` / `BILL_AMT` 这类大数值栏位不会压过 `PAY_0~PAY_6` 这类小数值栏位。
- **MLP 模型:** 3 层隐藏层 `[64, 32, 16]`，每层 ReLU + Dropout(0.2)，输出层为单一 logit(配合 `BCEWithLogitsLoss` 做二元分类)。
- **单笔 Validation 预测:** 脚本最后单独取 `X_val_scaled[0]` 丢进模型，印出原始特征值、真实标签、预测机率与预测标签。
- **训练与绘图:** 训练 50 epochs，每个 epoch 记录 train/val loss，结束后用 matplotlib 画出 Training vs Validation Loss 曲线并存成 `loss_curve.png`。
- **超参数说明:** 脚本最后会印出 Epoch、Batch Size、Learning Rate、Optimizer、隐藏层/神经

確認輸出結果正常 (gpt)

我加上 L2 early stopping 後 反而沒有變好？
我需要針對原始 MLP，至少加入一種模型改善策略，例如：Early Stopping、Dropout、L2 Regularization、調整 Learning Rate、更換 Optimizer、調整 Hidden Layer / Neuron

Show less ^

對，你這次的結果確實沒有改善，而且這是合理的結果。重點是「加入改善策略」不等於「指標一定會變好」。

你現在的結果：

指標	Baseline	Improved
Final Train Loss	0.4229	0.4299
Final Val Loss	0.4332	0.4325

Quiz3

初步架構建立 (claude)

Quiz3.py:

延續 Quiz 2_1 的信用卡違約預測資料集與 MLP 模型。

- 取得 MLP 對 Validation Dataset 輸出的 Prediction Score / Probability。
- 繪製 MLP 的 ROC Curve，X 軸為 FPR，Y 軸為 TPR。
- 計算 MLP 的 AUC，並將 AUC 數值標示於 ROC Curve。
- 簡要說明 ROC Curve 與 AUC 代表的意義。

使用與 Quiz 2 相同的 Training Dataset 與 Validation Dataset，選擇 SVM 或 Decision tree 或 Naïve Bayes 建立第二個模型。

- 取得第二個模型對 Validation Dataset 輸出的 Prediction Score / Probability。
- 將 MLP 與 第二個模型的 ROC Curve 繪製於同一張圖中。
- 分別計算兩個模型的 AUC，並將數值標示於圖中。
- 比較兩個模型的 ROC Curve 與 AUC，並說明哪一個模型具有較好的正負樣本區分能力。

5 hours ago

Created a file, edited a file, and 2 more tools >

Quiz3.py (语法检查通过) 延續 Quiz2-1 的資料與切分方式, 內容分兩部分:

Part A - MLP

- 優先載入 **Quiz2-1.py** 存下的 **quiz2_1_improved.pt** / **quiz2_1_scaler.pkl** (若找不到就用同樣設定重新訓練), 確保用的是同一個 improved MLP。
- 取得 validation set 的 sigmoid 機率分數, 用 **roc_curve** 算 FPR/TPR, **roc_auc_score** 算 AUC, 畫出單獨的 MLP ROC 曲線并把 AUC 標在圖例上, 存成 **roc_curve_mlp.png**。

Part B - 第二個模型(Decision Tree)

↓

確認圖片理解正確 (grmini)

所以 AUC = 0.77 代表模型隨機抽樣一個「真實違約客戶」與一個「真實未違約客戶」時，模型給予違約客戶較高預測風險？

沒錯！完全正確。

以這張圖來說， $AUC = 0.7727$ 代表：

如果你隨機抽出一名實際上會違約的客戶，以及一名實際上不會違約的客戶，模型預測出前者的違約風險分數高於後者的機率是 **77.27%**。

這也說明了這個 MLP 模型在區分「會違約」與「不會違約」的客戶上，具備相當不錯的鑑別與排序能力（顯然優於 50% 的隨機猜測）。